

AI-GraphDB-Engine SDK Documentation

Version: 1.0

For: LLMs, AI agents, and developers building graph-based memory systems

Table of Contents

1. [Overview](#)
2. [Installation](#)
3. [Core Concepts](#)
4. [Quick Start](#)
5. [API Reference](#)
 - [GraphStore](#)
 - [AgentMemory](#)
 - [Embedders](#)
 - [MCP Protocol](#)
 - [A2A Protocol](#)
 - [Orchestrator](#)
 - [ContextManager](#)
6. [Training Dataset Pipeline](#)
 - [DocumentLoader](#)
 - [DatasetBuilder](#)
 - [Output Formats](#)
 - [Ollama Q&A Generator](#)
 - [Export](#)
7. [Needle2 Integration](#)
 - [NeedleAgentGroup](#)
 - [Training Workflow](#)
 - [NeedleOrchestrator](#)
 - [build_needle_dataset\(\)](#)
 - [GRAPHDB_TOOL_SCHEMAS](#)
8. [JEPa-GraphRAG](#)
 - [Overview](#)
 - [Community Detection](#)
 - [GraphRAG Retrieval](#)
 - [Graph-JEPa World Model](#)
 - [Hybrid Search](#)
9. [JEPa-Needle Integration](#)
 - [JEPaNeedleAgent](#)
 - [Multi-mode Search](#)
 - [Community Tools](#)
 - [World-Model Training](#)
 - [JEPaOrchestrator](#)
10. [Usage Patterns](#)

- 11. [Examples](#)
- 12. [Best Practices](#)
- 13. [Troubleshooting](#)

Overview

The **AI-GraphDB-Engine** is a graph-based long-term memory system designed for AI agents and LLMs. It provides:

- **Unbounded memory** — store unlimited facts, conversations, and observations in a persistent graph
- **Label-scoped vector search** — retrieve only memories of the right type (avoids cross-type false matches)
- **Agent interoperability** — MCP (Agent→Tools&Data) and A2A (Agent→Agent) protocols
- **Context management** — stay within 16k-256k LLM context windows via smart retrieval + rolling summarization
- **Offline-first** — works with local Ollama models or cloud APIs; fully functional without internet

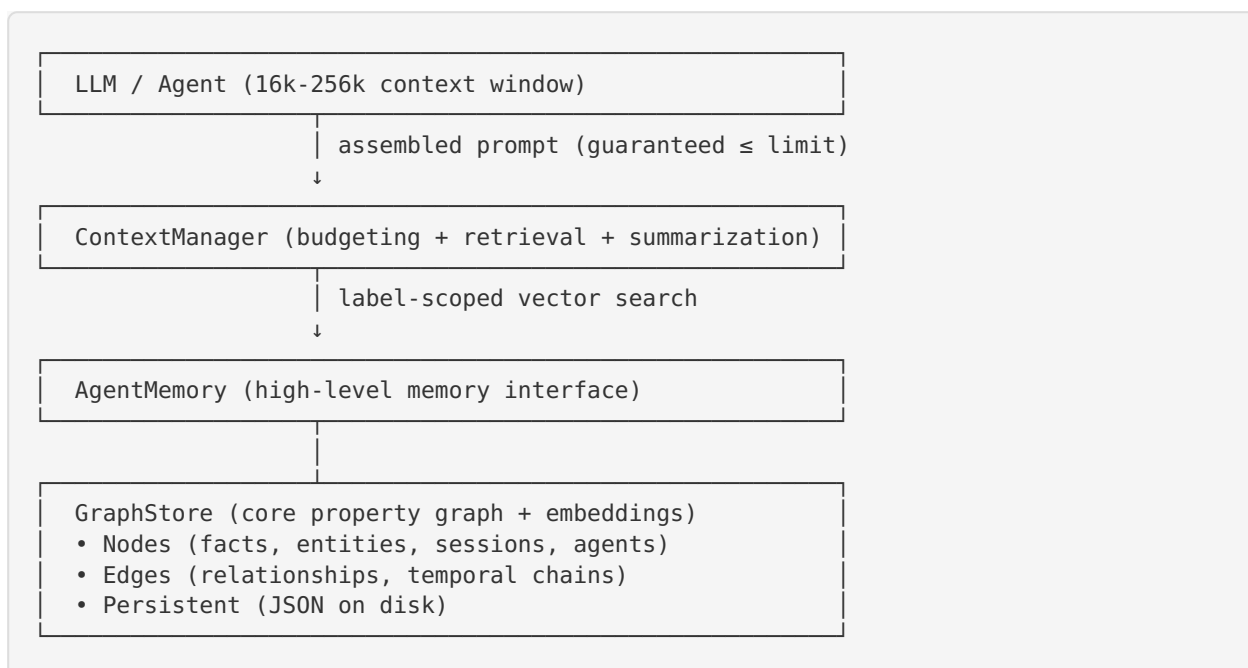
What Problem Does It Solve?

LLMs have a **fixed context window** (16k-256k tokens). Long conversations or large knowledge bases **overflow** and cause:

- Lost context (“the model forgot”)
- Errors (context limit exceeded)
- Expensive re-processing

This engine solves it by acting as **external long-term memory**: the graph holds everything, and only the most relevant memories are retrieved into the prompt each turn.

Architecture



Installation

Prerequisites

- Python 3.10+
- `uv` — recommended (fast, reproducible venvs)
- Optional: [Ollama](#) for local embedding / LLM models

Recommended — `uv venv`

`uv` is a drop-in replacement for `pip` / `venv` that is 10–100× faster and produces fully reproducible environments.

```
# 1. Install uv (once, system-wide)
curl -LsSf https://astral.sh/uv/install.sh | sh
# or: pip install uv

# 2. Clone the repository
git clone https://github.com/frederico-tech6868/ai-graph-db-engine.git
cd ai-graph-db-engine/python

# 3. Create an isolated virtual environment
uv venv .venv

# 4. Activate the environment
source .venv/bin/activate      # Linux / macOS
# .venv\Scripts\activate      # Windows PowerShell

# 5. Install dependencies
uv pip install -e .            # core engine — pure Python
uv pip install -r requirements.txt # fastapi, uvicorn, pydantic, numpy, pytest
```

Optional extras

```
bash
uv pip install -e ".[fast]"      # numpy acceleration for vector ops
uv pip install -e ".[test]"     # pytest test suite
```

Optional document-loader backends (install only what you need):

```
bash
uv pip install pypdf            # PDF support
uv pip install python-docx     # Word / DOCX support
uv pip install pandas openpyxl # CSV, Excel (.xlsx)
uv pip install pandas pyarrow  # Parquet export
```

Needle2 embedded agent (optional fine-tuning pipeline):

```
bash
uv pip install cactus-needle    # inference only
uv pip install 'cactus-needle[train]' # + fine-tuning (CPU / JAX)
uv pip install 'cactus-needle[train,gpu]' # NVIDIA GPU training
uv pip install 'cactus-needle[train,metal]' # Apple Silicon
```

Alternative — plain `pip`

```
python -m venv .venv
source .venv/bin/activate      # Linux / macOS
pip install -e .
pip install -r requirements.txt
```

Run the web UI

```
python -m webui.run           # → http://localhost:3000
```

Open <http://localhost:3000> and click **Seed demo** to populate example data.

The graph is persisted to `webui_graph.json` (override via `GRAPHDB_WEB_PATH`).

Run the tests

```
python -m pytest tests/ -v    # 156 tests, all should pass
```

Verify the install

```
from graphdb.store import GraphStore
from ai_memory.memory import AgentMemory
from ai_memory.embedder import LocalEmbedder

store = GraphStore()
memory = AgentMemory("test-agent", store=store, embedder=LocalEmbedder())
print("✅ AI-GraphDB-Engine is ready")
```

Core Concepts

1. Nodes

A node is an entity in the graph. Every node has:

- `id` (UUID, auto-generated)
- `label` (e.g., "Memory", "Entity", "Agent")
- `properties` (dict, e.g., {"text": "...", "memory_type": "fact"})
- `embedding` (optional vector for similarity search)
- `created_at` (timestamp)

Node labels in this system:

- `Memory` — a stored fact, observation, plan, reflection, or emotion
- `Entity` — a named entity (person, place, concept)
- `Session` — a conversation session
- `Agent` — an AI agent node
- `TOOL`, `RESOURCE`, `TOPIC` — MCP/A2A protocol nodes

2. Edges

An edge is a directed relationship between two nodes:

- `id` (UUID)

- `src_id` → `dst_id`
- `label` (e.g., "RELATES_TO", "REMEMBERS")
- `weight` (float, default 1.0)
- `properties` (dict)

Edge labels in this system:

- `RELATES_TO` — Memory → Entity
- `OCCURRED_IN` — Memory → Session
- `FOLLOWS` — Memory → Memory (temporal chain)
- `SIMILAR_TO` — Memory → Memory (semantic similarity)
- `KNOWS` — Agent → Entity
- `REMEMBERS` — Agent → Memory
- `EXPOSES` , `PROVIDES` , `INTERESTED_IN` , `PUBLISHED` , `SHARED_WITH` , `TAGGED` — MCP/A2A protocol edges

3. Label-Scoped Vector Search

The engine indexes embeddings **per label**. When you search for similar memories, it only compares against nodes with the **same label**, avoiding cross-type pollution (e.g., a query about “databases” won’t match an Entity named “Database Inc.”).

4. Memory Types

Memories are categorized by `memory_type` property:

- `observation` — raw input (user said X, assistant replied Y)
- `fact` — extracted knowledge
- `reflection` — higher-level summary
- `plan` — future intent
- `emotion` — affective state

5. Persistence

The graph is saved to disk as JSON. Load/save is explicit:

```
store = GraphStore(path="my_graph.json") # auto-loads if exists
# ... work ...
store.save() # persist to disk
```

Quick Start

Minimal Working Example

```
from graphdb.store import GraphStore
from ai_memory.memory import AgentMemory
from ai_memory.embedder import LocalEmbedder

# 1. Create a graph store (in-memory or persistent)
store = GraphStore()
embedder = LocalEmbedder() # offline, deterministic embedder

# 2. Create an agent memory
memory = AgentMemory(agent_id="my-agent", store=store, embedder=embedder)

# 3. Store some facts
memory.remember("The project uses a graph database with label-scoped vector search.")
memory.remember("Fred prefers Rust for performance-critical code.")
memory.remember("Deployment target is local Ollama, no cloud dependencies.")

# 4. Retrieve relevant memories
results = memory.recall("What database technology does the project use?", k=3)
for r in results:
    print(f"[{r.score:.3f}] {r.node.properties['text']}")

# 5. Persist to disk
store.save("agent_memory.json")
```

Output:

```
[0.712] The project uses a graph database with label-scoped vector search.
[0.301] Deployment target is local Ollama, no cloud dependencies.
[0.098] Fred prefers Rust for performance-critical code.
```

API Reference

GraphStore

Path: `graphdb.store.GraphStore`

The core property graph engine. Manages nodes, edges, indexes, and persistence.

Constructor

```
GraphStore(path: Optional[str] = None)
```

- `path` : If provided, auto-loads from disk on construction.

Methods

Node Operations

```
add_node(node: Node) -> Node
```

Add a node. Returns the same node (or existing if duplicate ID).

```
get_node(node_id: str) -> Optional[Node]
```

Retrieve a node by ID. Returns `None` if not found.

```
delete_node(node_id: str) -> None
```

Delete a node and **all connected edges** (cascade).

```
nodes_by_label(label: str) -> List[Node]
```

Get all nodes with a specific label.

```
all_nodes() -> List[Node]
```

Get all nodes in the graph.

Edge Operations

```
add_edge(edge: Edge) -> Edge
```

Add an edge. Both `src_id` and `dst_id` must exist.

```
get_edge(edge_id: str) -> Optional[Edge]
```

Retrieve an edge by ID.

```
delete_edge(edge_id: str) -> None
```

Delete an edge.

```
edges_from(node_id: str) -> List[Edge]
```

Get all outgoing edges from a node.

```
edges_to(node_id: str) -> List[Edge]
```

Get all incoming edges to a node.

```
edges_between(src_id: str, dst_id: str) -> List[Edge]
```

Get all edges from `src_id` to `dst_id`.

```
all_edges() -> List[Edge]
```

Get all edges in the graph.

Search

```
search_similar_nodes(
  query_vec: List[float],
  label: Optional[str] = None,
  k: int = 5
) -> List[Tuple[Node, float]]
```

Vector similarity search. If `label` is provided, search is **scoped** to that label.

Returns `[(node, score), ...]` sorted by score descending.

Persistence

```
save(path: Optional[str] = None) -> None
```

Save the graph to disk (JSON).

```
load(path: Optional[str] = None) -> None
```

Load the graph from disk.

Stats

```
stats() -> Dict[str, Any]
```

Returns node count, edge count, and label distribution.

AgentMemory

Path: `ai_memory.memory.AgentMemory`

High-level memory interface for a single agent. Wraps a `GraphStore` and provides semantic operations.

Constructor

```
AgentMemory(
  agent_id: str,
  store: GraphStore,
  embedder: Embedder,
  session_id: Optional[str] = None
)
```

- `agent_id` : Unique identifier for this agent
- `store` : The underlying graph store (shared across agents)
- `embedder` : Text→vector embedder
- `session_id` : Optional session scope

Methods

Store Memories

```
remember(
    text: str,
    memory_type: MemoryType = MemoryType.OBSERVATION,
    entities: Optional[List[str]] = None,
    session_id: Optional[str] = None
) -> RememberResult
```

Store a memory. Automatically:

- Embeds the text
- Checks for duplicates (cosine similarity > 0.95)
- Links to entities (creates them if needed)
- Chains to previous memories in session

Returns: RememberResult(memory_node, similar_existing, was_duplicate)

Memory types:

```
from ai_memory.schema import MemoryType

MemoryType.OBSERVATION    # raw input
MemoryType.FACT           # extracted knowledge
MemoryType.REFLECTION     # higher-level summary
MemoryType.PLAN           # future intent
MemoryType.EMOTION        # affective state
```

Retrieve Memories

```
recall(
    query: str,
    k: int = 5,
    memory_type: Optional[MemoryType] = None,
    session_id: Optional[str] = None
) -> List[RecalledMemory]
```

Label-scoped vector search over Memory nodes. Returns the k most relevant.

Returns: [RecalledMemory(node, score, context_snippet), ...]

```
get_context(query: str, k: int = 10) -> str
```

Formatted context string for LLM prompts (calls recall + formats).

Reflection

```
reflect(recent_k: int = 20) -> str
```

Synthesize a higher-level reflection from recent memories. Stores the reflection as a REFLECTION memory. Works offline (no LLM needed).

Entities

```
add_entity(
    name: str,
    entity_type: str = "concept",
    properties: Optional[Dict] = None
) -> Node
```

Create or retrieve an entity node.

```
get_entity(name: str) -> Optional[Node]
```

Get an entity by name.

Stats

```
stats() -> Dict[str, Any]
```

Returns counts by memory type, total entities, sessions, edges.

Embedders

Path: `ai_memory.embedder`

Text → vector transformations.

LocalEmbedder (default)

```
from ai_memory.embedder import LocalEmbedder

embedder = LocalEmbedder(dim=128)
vec = embedder.embed("hello world") # List[float] of length 128
```

- Deterministic, offline, no dependencies
- Hash-based character n-grams + tf-idf weighting
- Good for testing; weaker semantic understanding than neural models

OllamaEmbedder

```
from ai_memory.embedder import OllamaEmbedder

embedder = OllamaEmbedder(
    model="nomic-embed-text",
    host="http://localhost:11434"
)
vec = embedder.embed("hello world") # neural embedding
```

- Requires Ollama running locally: `ollama pull nomic-embed-text`
- Standard library only (uses `urllib`)
- Auto-discovers vector dimensionality on construction

Env vars:

- OLLAMA_HOST — default http://localhost:11434
- OLLAMA_EMBED_MODEL — default nomic-embed-text

OpenAIEmbedder

```
from ai_memory.embedder import OpenAIEmbedder

embedder = OpenAIEmbedder(model="text-embedding-3-small", api_key="sk-...")
```

- Requires openai package + API key
- Best semantic quality, but needs internet + costs money

Auto-Select

```
from ai_memory.embedder import get_embedder

# Order: Ollama → OpenAI → LocalEmbedder
embedder = get_embedder(prefer_ollama=True)

# Or via env var
# USE_OLLAMA=1 python script.py
embedder = get_embedder() # checks USE_OLLAMA
```

ollama_chat Helper

```
from ai_memory.embedder import ollama_chat

reply = ollama_chat(
    "Summarize the conversation",
    system="You are a note-taker.",
    model="llama3.1",
    host="http://localhost:11434"
)
```

Standard-library one-shot chat with local Ollama models. Useful for summarization.

MCP Protocol

Path: protocols.mcp

MCP = Model Context Protocol (Agent → Tools & Data). Exposes an agent's memory as callable tools and readable resources.

Build an MCP Server

```
from ai_memory.memory import AgentMemory
from protocols.mcp import build_memory_mcp_server

server = build_memory_mcp_server(memory, name="agent-mcp")
```

Tools exposed:

- remember_fact(text, memory_type?, entities?) → stores a memory

- `recall_memories(query, k?)` → retrieves relevant memories
- `search_nodes(text, label?, k?)` → vector search across any label
- `get_entity_info(name)` → entity + related memories
- `reflect(recent_k?)` → synthesize a reflection

Resources exposed:

- `memory://stats` — memory counts by type
- `memory://entities` — list of known entities
- `memory://recent` — most recent memories

MCP Client

```
from protocols.mcp import MCPClient

client = MCPClient()
client.connect(server)

# Call a tool
result = client.call_tool("remember_fact", {"text": "Paris is the capital of France."})
print(result) # MCP envelope: {"content": [...], "isError": False}

# Read a resource
stats = client.read_resource("memory://stats")
print(stats)
```

MCPTool / MCPResource

Define custom tools/resources:

```
from protocols.mcp import MCPServer, MCPTool, MCPResource

server = MCPServer(name="custom")

def my_tool_handler(args):
    return {"result": f"You said: {args['input']}"}

server.register_tool(
    name="echo",
    description="Echoes input",
    input_schema={"type": "object", "properties": {"input": {"type": "string"}}, "required": ["input"]},
    handler=my_tool_handler
)

def my_resource_reader():
    return {"status": "online"}

server.register_resource(
    uri="custom://status",
    name="System Status",
    mime_type="application/json",
    reader=my_resource_reader
)
```

A2A Protocol

Path: `protocols.a2a`

A2A = Agent-to-Agent. Interest-based memory routing. Agents advertise **interests** (topics), share memories, and the bus routes them to peers whose interests match.

Agent Card

```
from protocols.a2a import AgentCard, A2AAgent

card = AgentCard(
    agent_id="researcher",
    name="Research Agent",
    description="Gathers findings.",
    skills=["search", "summarize"],
    interests=["databases", "vector search", "AI"]
)

agent = A2AAgent(memory=memory, card=card)
```

A2A Bus

```
from protocols.a2a import A2ABus

bus = A2ABus(embedder=embedder, interest_threshold=0.35)
bus.register(agent)
```

Share Memory

```
result = bus.share_memory(
    sender_id="researcher",
    text="Label-scoped ANN search cut recall errors ~40%",
    topics=["databases", "vector search"],
    memory_type=MemoryType.FACT
)

print(result["delivered_to"]) # [{"agent_id": "engineer", "reason": "top-ic:databases", "score": 1.0}]
```

Routing logic:

1. **Topic overlap** — if any topic in `topics` matches any interest → deliver (score 1.0)
2. **Embedding similarity** — $\text{cosine}(\text{memory_vec}, \text{interest_vec}) \geq \text{interest_threshold}$ → deliver

Writes graph edges:

- `sender PUBLISHED memory_node`
- `memory_node TAGGED topic_node`
- `memory_node SHARED_WITH recipient`
- `recipient REMEMBERS memory_node`

Preview Interest

```
preview = bus.interested_agents(
    topics=["databases", "performance"],
    text="Benchmark results"
)
# [{"agent_id": "...", "reason": "...", "score": ...}, ...]
```

Direct Send

```
msg = bus.send(
    sender_id="agent-a",
    recipient_id="agent-b",
    content={"note": "Check this out"},
    type="text"
)
agent_b.receive(msg) # adds to inbox
```

Inbox

```
messages = agent.inbox # List[A2AMessage]
```

Orchestrator

Path: `protocols.orchestrator.Orchestrator`

Unified façade over a **shared** graph store + embedder. Manages multiple agents, their MCP servers, and the A2A bus.

Constructor

```
from protocols import Orchestrator

orch = Orchestrator(
    store=GraphStore(),
    embedder=LocalEmbedder(),
    interest_threshold=0.35
)
```

Create Agents

```
agent = orch.create_agent(
    agent_id="researcher",
    name="Research Agent",
    description="Gathers findings.",
    skills=["search"],
    interests=["databases", "AI"]
)
```

This creates:

- An `AgentMemory` with its own AGENT node

- An MCP server exposing that memory
- A2A registration

List Agents

```
agents = orch.agents()
# [{"agent_id": "...", "name": "...", "inbox": 0, "owned_memories": 5, "memory": {...}}, ...]
```

MCP Operations

```
# List tools
tools = orch.tools("researcher")

# Call a tool
result = orch.mcp_call("researcher", "remember_fact", {"text": "Rust is fast."})

# List resources
resources = orch.resources("researcher")

# Read a resource
data = orch.read_resource("researcher", "memory://stats")

# Call log
log = orch.call_log()
```

A2A Operations

```
# Share memory
result = orch.a2a_share(
    sender_id="researcher",
    text="Found a new optimization.",
    topics=["performance"]
)

# Direct send
msg = orch.a2a_send("agent-a", "agent-b", {"content": "hello"})

# Preview routing
preview = orch.preview_interest(["AI"], "Large language models")

# Message feed
feed = orch.messages(limit=50)

# Agent inbox
inbox = orch.inbox("researcher")
```

ContextManager

Path: `ai_memory.context_window.ContextManager`

Keeps a local LLM **within its context window** (16k-256k) by budgeting the prompt, retrieving only relevant memories, and rolling old turns into the graph.

Problem

Ollama models have a fixed `num_ctx` (16k-256k tokens). Long conversations overflow → the model “forgets” or errors.

Solution

The `ContextManager` sits between your model and the graph:

1. **Budgets** — reserves space for system prompt + model’s reply, splits the rest between memories and live transcript
2. **Retrieves** — only the top memories relevant to the current message (label-scoped vector search)
3. **Rolls up overflow** — when the transcript exceeds its sub-budget, oldest turns are summarized and written back into the graph as memories, then dropped from the prompt

Guarantee: assembled prompt $\leq$ `num_ctx` , always.

Constructor

```
from ai_memory.context_window import ContextBudget, ContextManager

budget = ContextBudget(
    context_limit=65_536,      # 64k; use 262_144 for 256k
    reserve_response=2_048,    # tokens for model's reply
    reserve_system=1_024,      # tokens for system prompt
    memory_fraction=0.5,       # 50% of working area → memories, 50% → transcript
    chars_per_token=4.0        # heuristic for default token counter
)

ctx = ContextManager(
    memory=memory,
    budget=budget,
    token_counter=None,        # optional: pass your own tokenizer
    summarizer=None           # optional: pass ollama_chat callback for better summaries
)
```

Usage Loop

```

system_prompt = "You are a helpful assistant with long-term memory."
history = [] # List[Dict[str, str]] with {"role": "user"/"assistant", "content": "..."}

while True:
    user_message = input("User: ")

    # Assemble a budget-safe prompt
    assembled, history = ctx.assemble(
        user_message,
        system_prompt,
        history,
        recall_k=12
    )

    # Check it fits
    print(f"Tokens: {assembled.total_tokens} / {budget.context_limit}")
    assert assembled.within_limit

    # Generate
    reply = my_ollama_generate(assembled.to_messages())
    print(f"Assistant: {reply}")

    # Persist the exchange
    ctx.ingest_turn(user_message, reply)

    # Continue the live transcript
    history.append({"role": "user", "content": user_message})
    history.append({"role": "assistant", "content": reply})

```

Breakdown

```

assembled.breakdown
# {
#   "system": 28,
#   "memories": 3200,
#   "history": 58000,
#   "user": 19,
#   "reserved_for_response": 2048,
#   "context_limit": 65536
# }

```

Custom Token Counter

```

import tiktoken

enc = tiktoken.get_encoding("cl100k_base")
ctx = ContextManager(
    memory=memory,
    budget=budget,
    token_counter=lambda text: len(enc.encode(text))
)

```

Custom Summarizer

```
from ai_memory.embedder import ollama_chat

def my_summarizer(raw_text: str) -> str:
    return ollama_chat(
        f"Summarize in 2-3 sentences:\n\n{raw_text}",
        system="You are a precise note-taker.",
        model="llama3.1"
    )

ctx = ContextManager(memory, budget, summarizer=my_summarizer)
```

JEPA-GraphRAG

Joint-Embedding Predictive Architecture + GraphRAG for advanced retrieval over knowledge graphs.

Overview

JEPA-GraphRAG combines two powerful paradigms:

1. **GraphRAG** — Community detection (Louvain algorithm), local search (k-hop expansion), and global search (community-based retrieval)
2. **Graph-JEPA** — A self-supervised world model that predicts latent graph states for energy-based retrieval

Key advantages:

- **Noise-resistant retrieval** — operates in abstract latent space, not raw text
- **Multi-hop reasoning** — predicts hidden relationships before traversing the graph
- **Pure Python** — works with `numpy` (optional `torch` acceleration)
- **No external dependencies** — no Neo4j, fully integrated with `GraphStore`

Community Detection

The `CommunityDetector` implements the Louvain algorithm to identify densely-connected clusters (communities) in your graph.

```

from graphdb import GraphStore
from ai_memory.jepa_graphrag import CommunityDetector

store = GraphStore()
# ... add nodes and edges ...

detector = CommunityDetector(store, resolution=1.0)
communities = detector.detect_communities() # {node_id: community_id}

# Group nodes by community
from collections import defaultdict
comm_groups = defaultdict(list)
for node in store.all_nodes():
    comm_id = communities.get(node.id)
    if comm_id is not None:
        comm_groups[comm_id].append(node)

print(f"Found {len(comm_groups)} communities")

```

Parameters:

- `resolution` (float, default 1.0) — higher values → more granular communities

GraphRAG Retrieval

The `GraphRAGRetriever` provides Microsoft GraphRAG-style retrieval over `GraphStore`.

Local Search (K-Hop Expansion)

Finds seed nodes via vector search, then expands their k-hop neighborhood:

```

from ai_memory.jepa_graphrag import GraphRAGRetriever
from ai_memory.embedder import LocalEmbedder

embedder = LocalEmbedder()
retriever = GraphRAGRetriever(store, embedder)

# Local search with 2-hop expansion
results = retriever.local_search(
    "machine learning engineer",
    k=5, # top-5 seed nodes
    max_hops=2, # expand 2 hops
    label="Person" # optional label filter
)

for node, score in results:
    print(f"{score:.3f} - {node.label}: {node.properties.get('name')}")

```

Global Search (Community-Based)

Scores entire communities, returning top-k communities with their member nodes:

```
# Global search across communities
results = retriever.global_search(
    "deep learning frameworks",
    k=3, # top-3 communities
    label="Framework" # optional
)

for comm_id, nodes, score in results:
    print(f"Community {comm_id} (score: {score:.3f})")
    for node in nodes[:5]:
        print(f"  - {node.label}: {node.properties.get('name')}")
```

Community Summaries

```
summary = retriever.get_community_summary(community_id=0)
print(summary)
# Output:
# Community 0:
#   12 nodes
#   - 5 Person nodes
#   - 4 Framework nodes
#   - 3 Language nodes
```

Graph-JEPA World Model

The core predictive component. Instead of directly matching text, JEPA:

1. Encodes queries & subgraphs into a **latent space**
2. **Predicts** the latent state of target structures
3. Retrieves via **energy minimization** (L2 distance in latent space)

Architecture

```
Context Encoder → Predictor → Predicted Latent State
                        ↓
                        L2 Distance
                        ↓
Target Encoder → Target Latent State (EMA)
```

- **Context Encoder** — processes queries/prompt text
- **Target Encoder** — processes graph substructures (updated via EMA)
- **Predictor** — maps context → predicted target (operates purely in latent space)
- **VICReg Loss** — prevents representation collapse (variance + covariance regularization)

Initialization

```
from ai_memory.jepa_graphrag import JEPAGraphRAG

jepa = JEPAGraphRAG(
    store=store,
    embedder=embedder,
    latent_dim=128, # latent space dimension
    use_torch=False # False=numpy, True=PyTorch (if installed)
)
```

Training

Train the world model on query-target pairs:

```
contexts = ["machine learning engineer", "systems programmer"]
targets = ["PyTorch deep learning", "Rust safety memory"]

loss_dict = jepa.train_step(
    contexts,
    targets,
    learning_rate=0.001,
    ema_alpha=0.99 # EMA momentum for target encoder
)

print(f"Loss: {loss_dict['total']:.4f}")
print(f" Similarity: {loss_dict['sim']:.4f}")
print(f" Variance: {loss_dict['var']:.4f}")
print(f" Covariance: {loss_dict['cov']:.4f}")
```

Loss components:

- **Similarity** — MSE between predicted and target latent states
- **Variance** — forces features to maintain variance (prevents collapse)
- **Covariance** — decorrelates feature dimensions

Latent Search

Precompute community embeddings for fast retrieval:

```
jepa.precompute_community_embeddings()
```

Search by **community** (fastest):

```
results = jepa.latent_search(
    "Python machine learning",
    k=3,
    mode='community'
)

for comm_id, distance in results:
    print(f"Community {comm_id}: distance={distance:.4f}")
```

Search by **node** (more granular):

```
results = jepa.latent_search(
    "Python machine learning",
    k=5,
    mode='node'
)

for node, distance in results:
    print(f"{node.label}: {node.properties.get('name')} (distance={distance:.4f})")
```

Hybrid Search

Combine **all retrieval modes** for maximum coverage:

```
results = jepa.hybrid_search(
    "Python deep learning engineer",
    k=3,
    use_local=True,      # k-hop expansion
    use_global=True,     # community retrieval
    use_latent=True      # energy-based JEPA search
)

# Results dictionary:
# {
#   'local': [(node, score), ...],
#   'global': [(comm_id, nodes, score), ...],
#   'latent_community': [(comm_id, distance), ...],
#   'latent_node': [(node, distance), ...]
# }

for node, score in results['local'][:3]:
    print(f"Local: {score:.3f} - {node.properties.get('name')}")

for comm_id, nodes, score in results['global'][:2]:
    print(f"Global: Community {comm_id} (score: {score:.3f})")

for comm_id, dist in results['latent_community'][:3]:
    print(f"Latent: Community {comm_id} (distance: {dist:.4f})")
```

Selective Hybrid Search

Disable specific modes:

```
results = jepa.hybrid_search(
    query,
    k=5,
    use_local=True,
    use_global=False,    # skip community search
    use_latent=False     # skip JEPA search
)

# Returns only: {'local': [...]}
```

Complete Example

```

from graphdb import GraphStore, Node, Edge
from ai_memory.embedder import LocalEmbedder
from ai_memory.jepa_graphrag import JEPAGraphRAG

# 1. Build graph
store = GraphStore()
embedder = LocalEmbedder()

alice = store.add_node(Node(
    label="Person",
    properties={"name": "Alice", "role": "ML Engineer"},
    embedding=embedder.embed("Alice is a machine learning engineer")
))

pytorch = store.add_node(Node(
    label="Framework",
    properties={"name": "PyTorch"},
    embedding=embedder.embed("PyTorch deep learning framework")
))

store.add_edge(Edge(src_id=alice.id, dst_id=pytorch.id, label="USES", weight=1.0))

# 2. Initialize JEPa-GraphRAG
jepa = JEPAGraphRAG(store, embedder, latent_dim=64, use_torch=False)

# 3. Train world model
contexts = ["machine learning engineer"]
targets = ["PyTorch deep learning"]

for epoch in range(5):
    loss_dict = jepa.train_step(contexts, targets)
    print(f"Epoch {epoch+1}: loss={loss_dict['total']:.4f}")

# 4. Precompute embeddings
jepa.precompute_community_embeddings()

# 5. Hybrid search
results = jepa.hybrid_search("Python ML engineer", k=3)

print("Local results:")
for node, score in results['local']:
    print(f"  {score:.3f} - {node.properties.get('name')}")

print("\nLatent results:")
for node, dist in results['latent_node']:
    print(f"  {dist:.4f} - {node.properties.get('name')}")

```

Backend Options

Numpy (default, lightweight)

- Pure Python + numpy
- No GPU required
- Suitable for small-medium graphs (< 10k nodes)

```
jepa = JEPAGraphRAG(store, embedder, use_torch=False)
```

PyTorch (optional, accelerated)

- Requires `pip install torch`
- GPU acceleration if available
- Suitable for large graphs (10k+ nodes)

```
jepa = JEPAGraphRAG(store, embedder, use_torch=True)
```

API Reference

CommunityDetector

```
CommunityDetector(store: GraphStore, resolution: float = 1.0)
```

Methods:

- `detect_communities()` -> `Dict[str, int]` — returns `{node_id: community_id}`

GraphRAGRetriever

```
GraphRAGRetriever(
    store: GraphStore,
    embedder: Optional[Embedder] = None,
    community_cache: Optional[Dict[str, int]] = None
)
```

Properties:

- `communities: Dict[str, int]` — cached community assignments

Methods:

- `local_search(query, k, max_hops, label)` -> `List[Tuple[Node, float]]`
- `global_search(query, k, label)` -> `List[Tuple[int, List[Node], float]]`
- `get_community_summary(community_id)` -> `str`
- `rebuild_communities()` -> `Dict[str, int]`

JEPAGraphRAG

```
JEPAGraphRAG(
    store: GraphStore,
    embedder: Optional[Embedder] = None,
    latent_dim: int = 128,
    use_torch: bool = None # auto-detects if not specified
)
```

Properties:

- `retriever: GraphRAGRetriever` — underlying GraphRAG retriever
- `use_torch: bool` — backend indicator

Methods:

- `train_step(contexts, targets, learning_rate, ema_alpha) -> Dict[str, float]`
- `precompute_community_embeddings() -> None`
- `latent_search(query, k, mode) -> List[Tuple[Union[int, Node], float]]`
- `hybrid_search(query, k, use_local, use_global, use_latent) -> Dict[str, Any]`

JEPA-Needle Integration

The JEPA-Needle integration marries the two subsystems above:

- **Needle2** (`ai_memory.needle_agent`) — compact, embedded function-calling agents whose tools call back into the graph.
- **JEPA-GraphRAG** (`ai_memory.jepa_graphrag`) — community detection plus local / global / latent / hybrid retrieval and a self-supervised Graph-JEPA world model.

Two classes bring them together:

- **JEPANeedleAgent** — a drop-in `NeedleAgentGroup` whose graph-backed tools route through a shared `JEPAGraphRAG` instance, upgrading the agent from plain similarity search to multi-mode retrieval, and adding community-level tools.
- **JEPA0rchestrator** — a `Needle0rchestrator` that routes queries by **JEPA-GraphRAG content relevance** (which group actually knows the most about the query) instead of matching the query against group names, with the Graph-JEPA latent energy used only to break near-ties.

Everything except live Needle inference (`.run()` / `.agent`) works without `cactus-needle` installed; the JEPA machinery needs only numpy (torch optional).

```
from ai_memory.jepa_needle import JEPANeedleAgent, JEPA0rchestrator
```

JEPANeedleAgent

```
from ai_memory.document_loader import DocumentLoader

# chunk_overlap MUST be < chunk_size
loader = DocumentLoader(chunk_size=200, chunk_overlap=40, min_chunk_len=20)

legal = JEPANeedleAgent("legal", loader=loader, default_search_mode="hybrid")
legal.ingest(["contract.pdf", "policy.docx"])
```

Constructor — same parameters as `NeedleAgentGroup` , plus:

Parameter	Default	Description
<code>latent_dim</code>	128	Graph-JEPA latent space dimensionality
<code>use_torch</code>	None	Force torch backend (None = auto-detect)
<code>default_search_mode</code>	"hybrid"	Mode used by <code>search_knowledge_base</code> when unspecified: <code>local</code> / <code>global</code> / <code>latent</code> / <code>hybrid</code>

If `tool_schemas` is omitted the agent uses `JEPA_TOOL_SCHEMAS` (see below).

The shared retriever/world-model is exposed lazily via the `.jepa` property and is always bound to the group's own `GraphStore` and embedder.

Multi-mode Search

`JEPANeedleAgent.search()` is the exact function the Needle

`search_knowledge_base` tool wraps — callable directly for programmatic use:

```
out = legal.search("termination clause", k=5, mode="hybrid", doc_type="")
# {"results": [{"text", "source", "doc_type", "node_id", "score"}, ...],
#  "count": 5, "mode": "hybrid"}
```

Mode	Backing call	Best for
<code>local</code>	<code>retriever.local_search</code> (k-hop)	pinpoint passages near the best match
<code>global</code>	<code>retriever.global_search</code> (communities)	broad / thematic questions
<code>latent</code>	<code>jepa.latent_search(mode="node")</code>	energy-based semantic recall
<code>hybrid</code>	fuses local + latent, de-duplicated	general default

Results are always sorted by descending score and capped at `k` ; an optional `doc_type` filters by document type.

Community Tools

Two extra tools are exposed to the agent (and callable directly):

```

legal.search_communities("contract obligations", k=3)
# {"communities": [{"community_id", "size", "score", "summary"}, ...], "count": 3}

legal.community_summary(2)
# {"community_id": 2, "summary": "Community 2: ..."}

```

World-Model Training

Self-supervised training of the Graph-JEPA world model using each node's text as context and its graph neighbours' text as targets (VICReg loss, EMA target encoder). Safe no-op on an empty graph.

```

history = legal.train_world_model(epochs=3) # -> [{"total", "sim", "var", "cov"}, ...]
legal.rebuild_index() # invalidate cached communities/latent after new ingest

```

JEPAOrchestrator

```

tech = JEPANeedleAgent("tech", loader=loader)
tech.ingest(["api_docs.md"])

orch = JEPAOrchestrator([legal, tech]) # use_latent=True, latent_margin=0.05
orch.route("JWT bearer token authentication").name # -> "tech"
orch.route("contract termination liability").name # -> "legal"

result = orch.run("What are the termination clauses?") # needs cactus-needle
# result["routed_to"] -> selected group name

```

Routing scores each group by its best JEPA-GraphRAG content match (retriever.local_search similarity). When two groups are within `latent_margin` of each other, the Graph-JEPA latent energy breaks the tie. If no group has matching content, it falls back to the base-class text-embedding router. Set `use_latent=False` to route on content only.

JEPA_TOOL_SCHEMAS

Needle-compatible tool schemas used by `JEPANeedleAgent` (max 5, Needle's limit):

Tool	Purpose
<code>search_knowledge_base</code>	multi-mode retrieval with a <code>mode</code> enum
<code>search_communities</code>	rank graph communities for a query
<code>get_community_summary</code>	summarize a community by id
<code>list_documents</code>	list ingested documents
<code>get_document_chunks</code>	fetch all chunks of one document

Key classes / functions:

- `JEPANeedleAgent(name, ..., latent_dim=128, use_torch=None, default_search_mode="hybrid")`

```

- JEPANeedleAgent.search(query, k, mode, doc_type) -> Dict[str, Any]
- JEPANeedleAgent.search_communities(query, k) -> Dict[str, Any]
-
JEPANeedleAgent.train_world_model(epochs, learning_rate, ema_alpha) -> List[Dict[str, float]]
- JEPAOrchestrator(groups, embedder=None, use_latent=True, latent_margin=0.05)
- JEPAOrchestrator.route(query) -> Optional[JEPANeedleAgent]
- JEPA_TOOL_SCHEMAS

```

Usage Patterns

Pattern 1: Single-Agent Long-Term Memory

```

from graphdb.store import GraphStore
from ai_memory.memory import AgentMemory
from ai_memory.embedder import get_embedder

store = GraphStore(path="agent.json")
memory = AgentMemory("assistant", store=store, embedder=get_embedder(prefer_ollama=True))

# Store facts as you learn them
memory.remember("User's name is Alice.")
memory.remember("Alice prefers Python over JavaScript.")

# Later: retrieve relevant context
context = memory.get_context("What programming language does the user prefer?", k=5)
print(context)

```

Pattern 2: Multi-Agent Collaboration via A2A

```

from protocols import Orchestrator

orch = Orchestrator()

# Create specialist agents
orch.create_agent("researcher", interests=["research", "data"])
orch.create_agent("engineer", interests=["code", "data"])
orch.create_agent("writer", interests=["content", "docs"])

# Researcher shares a finding
orch.a2a_share(
    "researcher",
    "Found benchmark data: 10x speedup with Rust.",
    topics=["data", "research"]
)
# → routed to "engineer" (has "data" interest)

# Engineer shares implementation
orch.a2a_share(
    "engineer",
    "Implemented the optimization in Rust.",
    topics=["code"]
)
# → routed to "writer" (no match) and self

# Check engineer's inbox
inbox = orch.inbox("engineer")
print(len(inbox), "messages")

```

Pattern 3: Budget-Safe Ollama Chat

```

from ai_memory.context_window import ContextBudget, ContextManager
from ai_memory.embedder import ollama_chat

budget = ContextBudget(context_limit=16_384) # 16k model
ctx = ContextManager(memory, budget)

history = []
for user_msg in ["Tell me about Python", "Now about Rust", "Compare them"]:
    assembled, history = ctx.assemble(user_msg, "You are helpful.", history)

    # Always fits
    assert assembled.within_limit

    # Generate
    reply = ollama_chat(assembled.to_prompt(), model="llama3.2:3b")

    # Persist + continue
    ctx.ingest_turn(user_msg, reply)
    history.extend([
        {"role": "user", "content": user_msg},
        {"role": "assistant", "content": reply}
    ])

```

Pattern 4: Project Knowledge Base

```
# Build a knowledge graph for a codebase
memory.remember("The API uses FastAPI and returns JSON.", memory_type=MemoryType.FACT)
memory.remember("Database is PostgreSQL, accessed via SQLAlchemy.", memory_type=MemoryType.FACT)
memory.remember("Auth is JWT-based, tokens expire in 1 hour.", memory_type=MemoryType.FACT)

# Add entities
memory.add_entity("FastAPI", entity_type="library")
memory.add_entity("PostgreSQL", entity_type="database")

# Query the knowledge base
results = memory.recall("What database do we use?", k=3)
for r in results:
    print(r.node.properties["text"])
```

Pattern 5: Session-Scoped Memories

```
# Start a session
session_id = "conv-2024-01-15"

memory.remember("User asked about deployment.", session_id=session_id)
memory.remember("I suggested Docker Compose.", session_id=session_id)

# Later: recall from this session only
results = memory.recall("What did we discuss?", session_id=session_id, k=5)
```

Examples

The repository includes working examples in `/examples/` :

File	Description
<code>example_mcp.py</code>	MCP tools & resources over graph memory
<code>example_a2a.py</code>	A2A interest-based memory routing
<code>example_orchestration.py</code>	Full MCP+A2A orchestration
<code>example_project_layers.py</code>	One agent per AI-layer workflow (Plan→Implement→Validate)
<code>example_ollama.py</code>	Use local Ollama models for embeddings + chat
<code>example_ollama_context.py</code>	Keep Ollama within 64k-256k context window

Run any example:

```
cd python
python examples/example_mcp.py
```

Best Practices

1. Choose the Right Embedder

Scenario	Embedder	Why
Offline development	LocalEmbedder	No dependencies, deterministic
Local deployment	OllamaEmbedder	Neural quality, no cloud
Production (cloud OK)	OpenAIEmbedder	Best quality

2. Use Memory Types Intentionally

- **OBSERVATION** — raw turn-level I/O
- **FACT** — extracted, deduplicated knowledge
- **REFLECTION** — periodically synthesize these for high-level summaries
- **PLAN** — for future intent
- **EMOTION** — for affective state (optional)

3. Persist Regularly

```
# Auto-load on startup
store = GraphStore(path="agent.json")

# Auto-save after critical operations
memory.remember("Important fact")
store.save()
```

4. Label-Scope Your Searches

Always pass `label` when you know the node type:

```
# Good: scoped
store.search_similar_nodes(vec, label="Memory", k=5)

# Bad: unscoped (searches across ALL node types)
store.search_similar_nodes(vec, k=5)
```

5. Tune `memory_fraction` for Your Context Window

Window	Recommended <code>memory_fraction</code>
16k	0.3–0.4 (favor live transcript)
64k	0.5 (balanced)
128k+	0.6–0.7 (favor retrieval)

6. Use `reflect()` Periodically

Every 20–50 turns, synthesize a reflection:

```
summary = memory.reflect(recent_k=30)
print(summary)
```

This keeps high-level context available for retrieval.

7. Monitor Duplicate Detections

```
result = memory.remember("Paris is the capital of France.")
if result.was_duplicate:
    print("Already knew this fact.")
```

8. Delete Stale Sessions

```
# Delete old session nodes + their memories
session_nodes = store.nodes_by_label("Session")
for s in session_nodes:
    if s.properties.get("created_at") < cutoff_timestamp:
        store.delete_node(s.id) # cascades to connected memories
```

Troubleshooting

“Connection refused” when using `OllamaEmbedder`

Cause: Ollama server not running.

Fix:

```
ollama serve &
ollama pull nomic-embed-text
```

“`GraphError: DimensionMismatch`”

Cause: Embeddings have different dimensions (e.g., switched embedders mid-session).

Fix: Use the same embedder consistently, or clear the graph and rebuild.

Context still overflows with ContextManager

Cause: Your actual `num_ctx` is smaller than `context_limit`, or you're using a real tokenizer and the heuristic (`chars/4`) underestimated.

Fix:

1. Check Ollama's actual `num_ctx`: `ollama show <model> | grep num_ctx`
2. Pass a real tokenizer as `token_counter`
3. Lower `context_limit` by 10-20% as a safety margin

A2A routing doesn't deliver messages

Cause: No topic overlap and embedding similarity below `interest_threshold`.

Fix:

1. Check agent interests: `agent.card.interests`
2. Lower `interest_threshold` (default 0.35, try 0.25)
3. Use explicit topic overlap (more reliable than embedding similarity with `LocalEmbedder`)

Graph file corrupted after crash

Cause: Interrupted write.

Fix: The Rust backend uses atomic writes (`.tmp` → `rename`), so corruption is rare. The Python backend doesn't — use the Rust backend for production, or implement your own backup strategy.

Memory search returns irrelevant results

Cause: `LocalEmbedder` is weak for semantic similarity.

Fix: Switch to `OllamaEmbedder` or `OpenAIEmbedder`:

```
memory = AgentMemory("agent", store, embedder=OllamaEmbedder())
```

WebUI shows 0 agents after restart

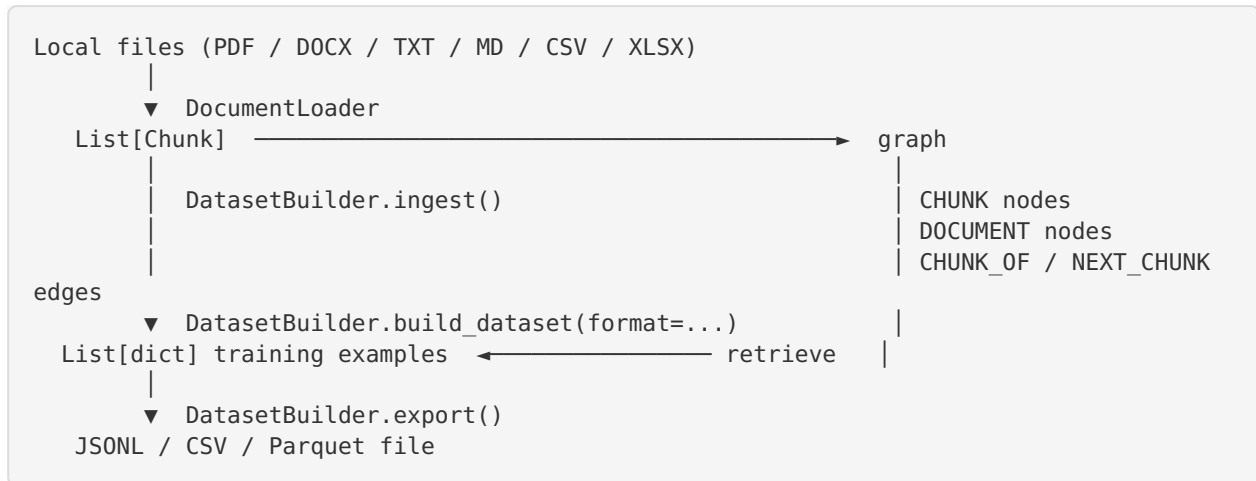
Cause: Agents are not persisted separately; only the graph is.

Fix: The webui's `service.py` calls `_rehydrate_agents()` which recreates agents from `AGENT` nodes in the graph. Make sure you're using the same `webui_graph.json` path.

Training Dataset Pipeline

The `ai_memory` package ships a complete **document-to-training-data** pipeline that turns local files into fine-tuning datasets for any LLM.

Pipeline overview



Quick start

```

from ai_memory.dataset_builder import DatasetBuilder
from ai_memory.embedder import LocalEmbedder
from graphdb.store import GraphStore

store = GraphStore(path="my_docs.json") # persist to disk (optional)
builder = DatasetBuilder(store=store, embedder=LocalEmbedder())

# 1 – Ingest documents
result = builder.ingest([
    "paper.pdf",
    "notes.md",
    "data.csv",
    "sheet.xlsx",
])
print(result) # IngestResult(docs=3, chunks=42, skipped=0)

# 2 – Build a dataset (vector-search scoped if query given)
dataset = builder.build_dataset(format="alpaca", k=500)

# 3 – Export
builder.export(dataset, "train.jsonl", file_format="jsonl")

```

DocumentLoader

`ai_memory.document_loader.DocumentLoader`

Parses a local file into a list of `Chunk` objects. Format is inferred from the file extension.

Supported formats

Extension	Library required
<code>.txt</code> , <code>.md</code> , <code>.markdown</code>	built-in
<code>.csv</code>	built-in <code>csv</code>
<code>.pdf</code>	<code>pip install pypdf</code>
<code>.docx</code> , <code>.doc</code>	<code>pip install python-docx</code>
<code>.xlsx</code> , <code>.xls</code>	<code>pip install pandas openpyxl</code>

Constructor

```
DocumentLoader(
    chunk_size: int = 800,      # max chars per chunk
    chunk_overlap: int = 100,   # overlap between adjacent chunks
    min_chunk_len: int = 60,    # discard chunks shorter than this (text only)
)
```

Note: `min_chunk_len` is bypassed for CSV rows — every non-empty row is always a valid chunk regardless of its character count.

Methods

```
loader.load(path: str) -> List[Chunk]
```

Returns a list of `Chunk` dataclass instances:

```
@dataclass
class Chunk:
    text: str
    chunk_index: int
    source_path: str
    doc_type: str      # "pdf" | "docx" | "txt" | "md" | "csv" | "excel"
    page: Optional[int] # PDF only (1-based)
    section: Optional[str] # DOCX heading / Excel sheet name
    row: Optional[int]   # CSV / Excel row number
    metadata: dict
```

Chunking strategy

Text is split on double-newlines (paragraph boundaries). Paragraphs are merged up to `chunk_size`, then split with a `chunk_overlap`-character sliding window for large paragraphs. Paragraphs shorter than `min_chunk_len` are discarded as noise (except CSV rows — see above).

DatasetBuilder

```
ai_memory.dataset_builder.DatasetBuilder
```

Orchestrates ingest → graph storage → dataset generation → export.

Constructor

```
DatasetBuilder(
    store: GraphStore = None,                # defaults to a fresh in-memory store
    embedder: Embedder = LocalEmbedder(),      # text-to-vector
    loader: DocumentLoader = DocumentLoader(), # file parser
    qa_generator: Callable[[str], str] = None, # question generator (offline
    heuristic by default)
    system_prompt: str = "...",              # system message for openai format
)
```

Graph schema created by ingest

Node label	Key properties
Document	source_path, filename, doc_type, num_chunks, ingested_at
Chunk	text, chunk_index, source_path, doc_type, page, section, row

Edge label	Meaning
CHUNK_OF	Chunk → Document
NEXT_CHUNK	Chunk → next Chunk in the same document

ingest(paths)

```
result: IngestResult = builder.ingest(["doc.pdf", "notes.md"])
# result.documents - new documents added
# result.chunks_stored - total chunk nodes created
# result.chunks_skipped - chunks skipped (duplicate document)
# result.files - list of ingested file paths
# result.errors - dict {path: error_message} for failed files
```

Already-ingested documents (same absolute path) are **skipped** automatically. Delete the Document node if you want to re-ingest.

build_dataset(format, source_paths, query, k, system_prompt)

```
dataset: List[dict] = builder.build_dataset(
    format="alpaca",                # see Output Formats section
    source_paths=["doc.pdf"],       # optional: restrict to specific files
    query="vector similarity",      # optional: retrieve by semantic similarity
    k=500,                          # max training examples
)
```

stats()

```
st = builder.stats()
# {"documents": 3, "chunks": 42, "chunks_by_type": {"txt": 15, "csv": 3}, ...}
```

list_documents()

```
docs = builder.list_documents()
# [{"filename": "paper.pdf", "source_path": "...", "doc_type": "pdf", "num_chunks": 12, ...}]
```

Output Formats

Five training-data formats are supported:

"raw" — plain chunk text

```
{
  "text": "Graph databases store data as nodes and edges.",
  "source": "/abs/path/notes.txt",
  "chunk_index": 0,
  "page": null,
  "section": null,
  "doc_type": "txt"
}
```

"completion" — prompt / completion pair (classic fine-tuning)

```
{
  "prompt": "Based on the following document excerpt, continue or summarise...\n\n<chunk text>",
  "completion": "<chunk text>"
}
```

"qa" — question / answer / context triple

```
{
  "question": "What does the document say about Graph databases?",
  "answer": "Graph databases store data as nodes and edges.",
  "context": "Graph databases store data as nodes and edges.",
  "source": "/abs/path/notes.txt",
  "page": null,
  "section": null
}
```

"alpaca" — Alpaca / LLaMA instruction format

```
{
  "instruction": "What does the document say about Graph databases?",
  "input": "Graph databases store data as nodes and edges.",
  "output": "Graph databases store data as nodes and edges."
}
```

"openai" — OpenAI chat fine-tuning format

```
{
  "messages": [
    {"role": "system", "content": "You are a knowledgeable assistant..."},
    {"role": "user", "content": "What does the document say about Graph data-bases?"},
    {"role": "assistant", "content": "Graph databases store data as nodes and edges."}
  ]
}
```

Ollama Q&A Generator

By default, questions are generated with a **fast offline heuristic** (no model needed). To generate higher-quality, natural-language questions using a local Ollama model, pass `ollama_qa_generator()` :

```
from ai_memory.dataset_builder import DatasetBuilder, ollama_qa_generator

builder = DatasetBuilder(
    store=store,
    embedder=embedder,
    qa_generator=ollama_qa_generator(
        model="llama3.2:3b",          # any Ollama model
        host="http://localhost:11434",
    ),
)
```

You can also pass any `Callable[[str], str]` as `qa_generator` for a custom generator:

```
def my_generator(text: str) -> str:
    return f"Can you explain: {text[:80]}?"

builder.qa_generator = my_generator
```

Export

```
builder.export(
    dataset: List[dict],      # output of build_dataset()
    output_path: str,         # file path to write
    file_format: str,         # "jsonl" (default) | "csv" | "parquet"
)
```

Format	Notes
jsonl	One JSON object per line — works directly with OpenAI fine-tuning and HuggingFace <code>datasets.load_dataset("json", ...)</code>
csv	Flat CSV — nested structures (e.g. messages list in openai format) are JSON-serialised into the cell
parquet	Requires <code>pip install pandas pyarrow</code>

Full example — alpaca JSONL

```
from ai_memory.dataset_builder import DatasetBuilder
from ai_memory.embedder import LocalEmbedder
from graphdb.store import GraphStore

builder = DatasetBuilder(store=GraphStore(), embedder=LocalEmbedder())
builder.ingest(["paper.pdf", "notes.md", "faq.csv"])

dataset = builder.build_dataset(format="alpaca", k=1000)
path = builder.export(dataset, "alpaca_train.jsonl")
print(f"Wrote {len(dataset)} examples to {path}")
```

See `examples/example_training_dataset.py` for a complete end-to-end walkthrough including all 5 formats and query-scoped retrieval.

Needle2 Integration

Needle2 (pip: `cactus-needle`) is a compact **embedded function-calling model** — not a chat LLM. Every response is a structured JSON tool call (or `[]` for off-topic queries). It is fine-tuned via LoRA adapters on JSONL data and exported to a `.cact` archive.

This engine integrates Needle2 as a **trainable query layer** over the graph knowledge base:

- Each **knowledge group** owns its own graph partition, ingests its own documents, and trains its own Needle adapter.
- Needle's tools call **directly back into the graph** (`search_knowledge_base` , `list_documents` , `get_document_chunks`).
- An **orchestrator** routes each query to the most relevant group.

The ingest / export / routing / stats surface works **without** `cactus-needle` installed. Only live inference (`group.agent` , `group.run()`) requires it.

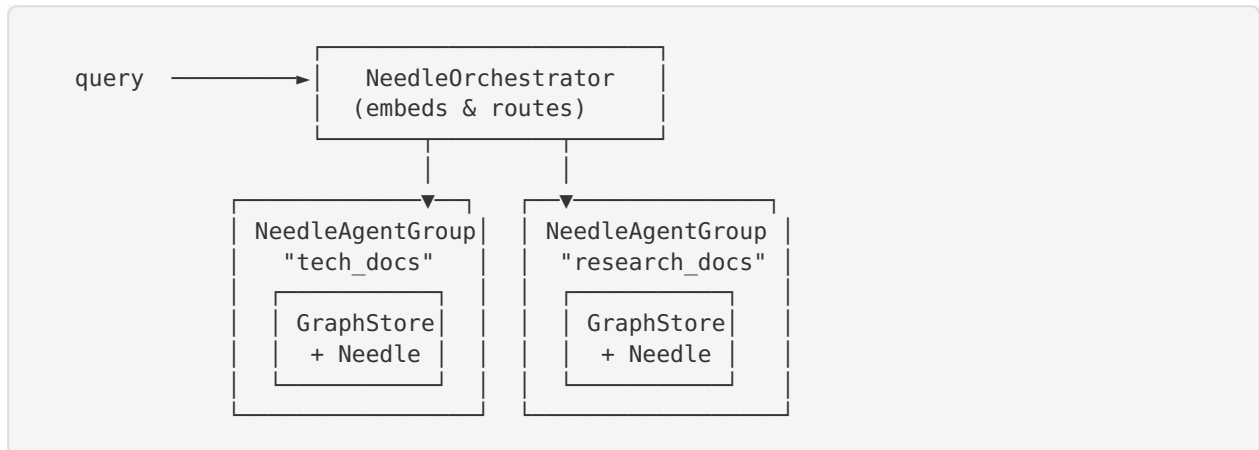
Installation

```

pip install cactus-needle                # inference only
pip install 'cactus-needle[train]'      # + fine-tuning (CPU/JAX)
pip install 'cactus-needle[train,gpu]'  # NVIDIA GPU training
pip install 'cactus-needle[train,metal]' # Apple Silicon

```

Grouped knowledge bases



NeedleAgentGroup

```
ai_memory.needle_agent.NeedleAgentGroup
```

A named, trainable knowledge group backed by the graph and Needle2.

Constructor

```

NeedleAgentGroup(
    name: str,                # logical group name, e.g. "legal"
    store: GraphStore = None, # defaults to a fresh in-memory store
    embedder: Embedder = None, # defaults to LocalEmbedder
    loader: DocumentLoader = None, # defaults to DocumentLoader()
    system: str = None,        # system string (default "knowledge_group:
    <name>")
    weights: str = None,       # optional path to a .cact archive
    tool_schemas: List[dict] = None, # override GRAPHDB_TOOL_SCHEMAS (max 5)
)

```

Methods

```

group.ingest(paths: List[str])           # → IngestResult
group.export_training_data(path, k=500, off_topic_ratio=0.125) # → written path
group.load_weights(weights_path: str)    # load a .cact archive (invalidates
agent cache)
group.agent                             # → needle.Needle (lazy; raises Import-
Error if not installed)
group.run(query, max_steps=8)            # → Needle response dict
group.complete(text)                     # → single-turn Needle response
group.reset()                            # rewind conversation history
group.stats()                            # → {documents, chunks, group_name,
weights, tool_count, ...}

```

Training Workflow

```

from ai_memory.needle_agent import NeedleAgentGroup

# 1 – Ingest documents into the group's graph
group = NeedleAgentGroup("legal", system="knowledge_group: legal")
group.ingest(["contract.pdf", "policy.docx"])

# 2 – Export Needle fine-tuning JSONL (positive + off-topic examples)
group.export_training_data("legal_train.jsonl", k=500)

```

```

# 3 – Fine-tune Needle outside Python
needle finetune legal_train.jsonl --epochs 20 --out legal_adapter.pkl
needle build checkpoints/needle2.pkl --lora legal_adapter.pkl --out legal.cact

```

```

# 4 – Load the trained weights and query
group.load_weights("legal.cact")
result = group.run("What are the termination clauses?")
print(result["function_calls"])

```

NeedleOrchestrator

```
ai_memory.needle_agent.NeedleOrchestrator
```

Routes queries across multiple `NeedleAgentGroup` instances by embedding the query and comparing it to each group's name/system string.

Constructor

```

NeedleOrchestrator(
    groups: List[NeedleAgentGroup] = None,
    embedder: Embedder = None,      # defaults to LocalEmbedder
)

```

Methods

```

orch.add_group(group)                # register a group
orch.route(query) -> NeedleAgentGroup # pick the best group (None if empty)
orch.run(query, max_steps=8)          # route + run; adds "routed_to" to the
result                                #
orch.export_all_training_data(out_dir, k=500) # → {group_name: path}
orch.stats()                            # → {group_name: stats}

```

```

from ai_memory.needle_agent import NeedleOrchestrator

orch = NeedleOrchestrator(groups=[tech_group, research_group])
orch.export_all_training_data("./training") # tech_docs_train.jsonl, ...

result = orch.run("How does vector search work?")
print(result["routed_to"]) # "tech_docs"

```

build_needle_dataset()

```

DatasetBuilder.build_needle_dataset(tools, k=500, off_topic_ratio=0.125, source_paths=None,
system=None)

```

Generates the Needle2 JSONL examples directly (used internally by `export_training_data`). Each example maps a natural-language query to the expected tool call:

```

{
  "query": "What does the document say about Vector Search?",
  "tools": [ /* full JSON schema list, embedded verbatim */ ],
  "answers": [
    { "name": "search_knowledge_base", "arguments": { "query": "Vector Search", "k": 5 } }
  ],
  "reasoning": "'Vector Search' -> query; default k=5",
  "system": "knowledge_group: tech"
}

```

Off-topic examples (interleaved at `off_topic_ratio` $\approx$ 1-in-8) carry an empty `answers` list so the model learns **not** to call a tool on unrelated queries:

```

{
  "query": "What is the current weather in London?",
  "tools": [ /* ... */ ],
  "answers": [],
  "reasoning": "query is not answerable by any declared tool"
}

```

Parameter	Meaning
<code>tools</code>	List of Needle-compatible JSON schema dicts, embedded in every example
<code>k</code>	Max positive (non-off-topic) examples
<code>off_topic_ratio</code>	Fraction of off-topic examples (default 0.125 $\approx$ 1-in-8). <code>0.0</code> disables them
<code>source_paths</code>	Restrict to chunks from these files
<code>system</code>	Optional system string injected into every example

GRAPHDB_TOOL_SCHEMAS

`ai_memory.needle_agent.GRAPHDB_TOOL_SCHEMAS` — three built-in, Needle-compatible tool schemas backed by the graph:

Tool	Purpose
<code>search_knowledge_base</code>	Vector search for top-k relevant chunks (<code>query</code> , <code>k</code> , <code>doc_type</code>)
<code>list_documents</code>	List all ingested documents with metadata
<code>get_document_chunks</code>	Retrieve all chunks from one document by <code>source_path</code>

Needle allows **max 5 tools** in context per query — these three leave room for custom additions via the `tool_schemas` parameter.

See `examples/example_needle_agent.py` for a complete end-to-end walkthrough (works without `cactus-needle` installed).

Summary

You now have everything you need to integrate the **AI-GraphDB-Engine** into your LLM or agent system:

1. **Install** — `pip install -e .` in `python/`
2. **Store** — `memory.remember(text)`
3. **Retrieve** — `memory.recall(query, k=5)`
4. **Stay in context** — `ContextManager` for budget-safe prompts
5. **Collaborate** — `Orchestrator` + A2A for multi-agent systems

6. **Expose tools** — MCP servers for tool-calling LLMs
7. **Persist** — `store.save(path)`
8. **Train** — `DatasetBuilder` to turn local docs into fine-tuning datasets
9. **Needle2** — `NeedleAgentGroup` + `NeedleOrchestrator` for trainable embedded function-calling agents
10. **JEPA-Needle** — `JEPANeedleAgent` + `JEPAOrchestrator` give Needle agents JEPA-GraphRAG multi-mode retrieval and content-relevance routing

The graph holds **everything**; retrieval surfaces **only what's relevant**. Your LLM never overflows, and nothing is lost.

For more details, see:

- `examples/` — working code samples
- `protocols/` — MCP/A2A source
- `ai_memory/` — memory + context management source
- `graphdb/` — core graph engine

License: MIT

Repository: <https://github.com/frederico-tech6868/ai-graph-db-engine>

Issues: <https://github.com/frederico-tech6868/ai-graph-db-engine/issues>

Happy building! 🚀